

DOC PROYECTO 3

Daniel Manzanera, 202411443, d.manzanerat@uniandes.edu.co

Santiago Gomez, 202221885, s.gomezp2@uniandes.edu.co

Santiago Pinilla, 202315246, s.pinillap2@uniandes.edu.co

Índice

Índice	1
Diseño general del sistema	2
Visión general	2
Cambios realizados	2
Diagrama de de alto nivel	2
Diagrama de bajo nivel	4
Diagramas de secuencia	5
Diagramas de casos de Uso	8
Justificación	11
Organización Lógica y Claridad	12
Uso de Herencia y Polimorfismo	12
Encapsulamiento y Control de Errores	12
Gestión de Reglas de Negocio	13
Diseño Orientado a Casos de Uso	13
Extensibilidad y Mantenibilidad	13
Diseño de la Interfaz Gráfica	13
Mockups	14
Implementación	15

Diseño general del sistema

(Los diagramas en alta resolución se encuentran en el repositorio para su mejor visualización).

Visión general

El sistema ofrece una interfaz modular grafica desde la cual distintos roles interactúan con el marketplace para publicar, negociar, comprar y administrar tiquetes de diversos eventos. Los roles principales son: Cliente, Organizador y Administrador.

- **Interfaz gráfica:** interfaz gráfica por roles del usuario, y maneja las entradas/salidas del usuario para que este pueda acceder, consultar y actualizar los datos de marketplace y persistencia a través de la lógica del sistema.
- **marketplace:** contiene la lógica de negocio relacionada para publicar, listar, negociar y concretar operaciones sobre tickets. Incluye clases como Marketplace (fachada), Oferta y ContraOferta, y controla estados y validaciones del flujo (por ejemplo, que un ticket sea transferible y no esté ya transferido).
- **persistencia:** provee la carga y guardado del estado del sistema.
- **usuarios:** define los roles y sus permisos: Cliente (compra y puede publicar ofertas), Organizador (crea/administra eventos) y Administrador (supervisa y aplica acciones de control).

- Diagrama de classes de projeto para o sistema de gerenciamento de eventos, seguindo o padrão MVC e utilizando o ORM JPA.

Entidades (Classes de Modelo):

 - Usuario**: Representa o usuário do sistema. Campos: login (String), senha (String), email (String), nome (String), dataNascimento (Date), sexo (String), endereco (String), telefone (String), dataCadastro (Date). Métodos: login(), logout(), cadastrar(), atualizar(), buscar().
 - Evento**: Representa o evento. Campos: nome (String), data (Date), local (String), descricao (String), preco (Double), status (String). Métodos: criar(), atualizar(), buscar(), cancelar().
 - Organizador**: Representa o organizador do evento. Campos: nome (String), email (String), telefone (String), dataCadastro (Date). Métodos: criar(), atualizar(), buscar().
 - Participante**: Representa o participante do evento. Campos: nome (String), email (String), telefone (String), dataCadastro (Date). Métodos: criar(), atualizar(), buscar().
 - EventoOrganizador**: Associação entre Evento e Organizador. Campos: evento (Evento), organizador (Organizador).
 - EventoParticipante**: Associação entre Evento e Participante. Campos: evento (Evento), participante (Participante).

Controle (Classes de Controle):

 - EventoControlador**: Controla as operações relacionadas aos eventos. Métodos: criarEvento(), atualizarEvento(), buscarEventos(), cancelarEvento().
 - OrganizadorControlador**: Controla as operações relacionadas aos organizadores. Métodos: criarOrganizador(), atualizarOrganizador(), buscarOrganizadores().
 - ParticipanteControlador**: Controla as operações relacionadas aos participantes. Métodos: criarParticipante(), atualizarParticipante(), buscarParticipantes().

Visão (Classes de Visão):

 - EventoForm**: Interface para a criação e edição de eventos. Campos: nome (String), data (Date), local (String), descricao (String), preco (Double), status (String). Métodos: criarEvento(), atualizarEvento().
 - OrganizadorForm**: Interface para a criação e edição de organizadores. Campos: nome (String), email (String), telefone (String), dataCadastro (Date). Métodos: criarOrganizador(), atualizarOrganizador().
 - ParticipanteForm**: Interface para a criação e edição de participantes. Campos: nome (String), email (String), telefone (String), dataCadastro (Date). Métodos: criarParticipante(), atualizarParticipante().

Utilitários (Classes de Utilitários):

 - Utilitario**: Classe utilitária que contém métodos auxiliares. Métodos: validarEmail(), validarSenha(), validarData(), validarLocal(), validarPreco(), validarStatus().

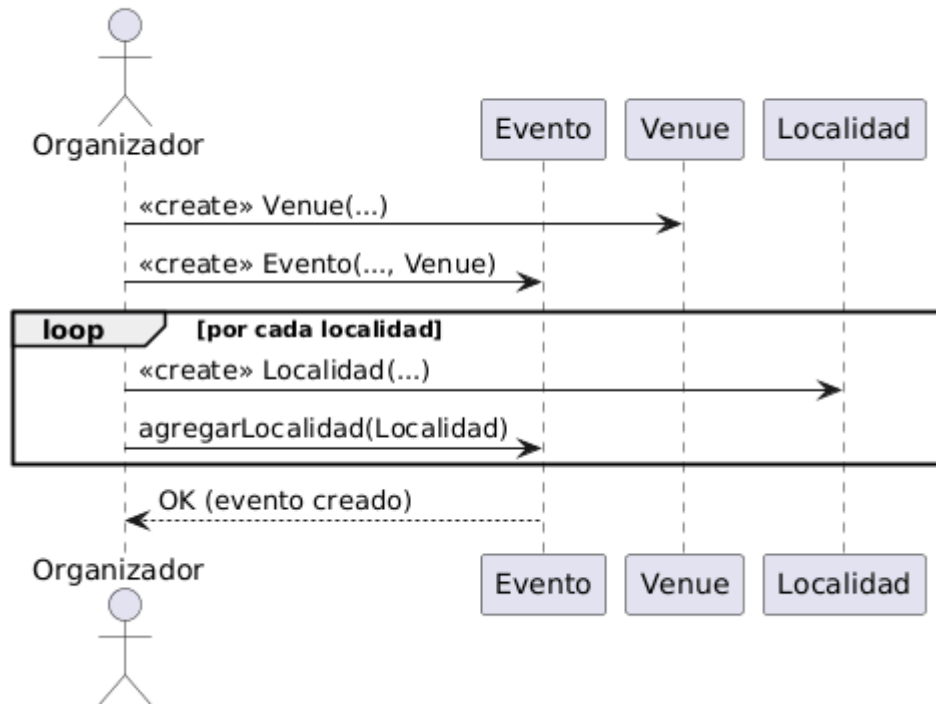
Diagrama de Classes:

```
classDiagram
    class Usuario {
        login: String
        senha: String
        email: String
        nome: String
        dataNascimento: Date
        sexo: String
        endereco: String
        telefone: String
        dataCadastro: Date
    }
    class Evento {
        nome: String
        data: Date
        local: String
        descricao: String
        preco: Double
        status: String
    }
    class Organizador {
        nome: String
        email: String
        telefone: String
        dataCadastro: Date
    }
    class Participante {
        nome: String
        email: String
        telefone: String
        dataCadastro: Date
    }
    class EventoOrganizador {
        evento: Evento
        organizador: Organizador
    }
    class EventoParticipante {
        evento: Evento
        participante: Participante
    }

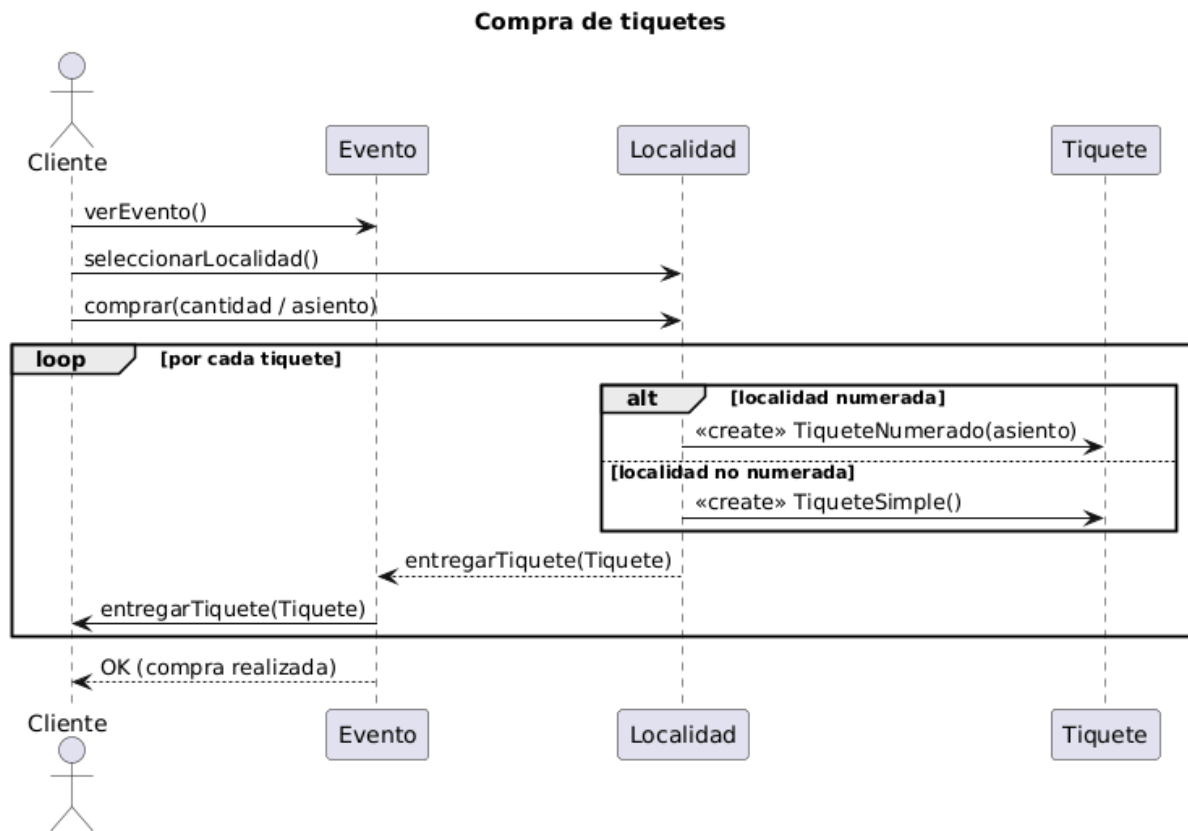
    Usuario "1" -- "1" EventoOrganizador
    Usuario "1" -- "1" EventoParticipante
    Evento "1" -- "1" Organizador : EventoOrganizador
    Evento "1" -- "1" Participante : EventoParticipante
    Organizador "1" -- "1" EventoOrganizador
    Participante "1" -- "1" EventoParticipante
```

Diagramas de secuencia

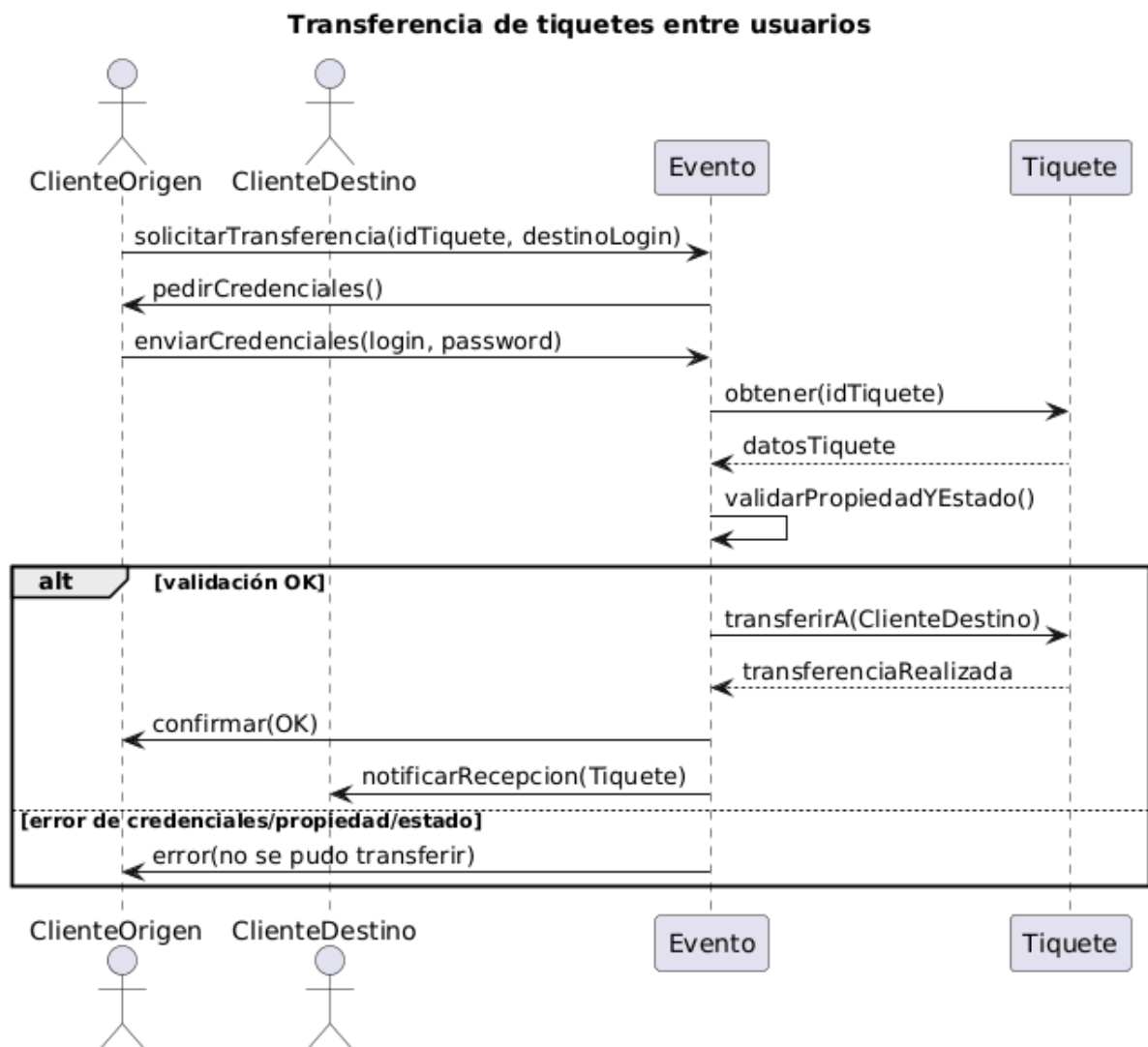
Crear evento y asignar localidades



Iniciamos con el diagrama de secuencia para la creación de un evento, porque el evento es el objeto que dispara la interacción con el organizador y luego sirve de base para las demás operaciones. El flujo es: se crea el Venue, con ese venue se crea el Evento, y en un bucle por cada localidad se crea la Localidad y se llama a `agregarLocalidad(Localidad)` sobre el evento. Al final el sistema devuelve OK (evento creado).

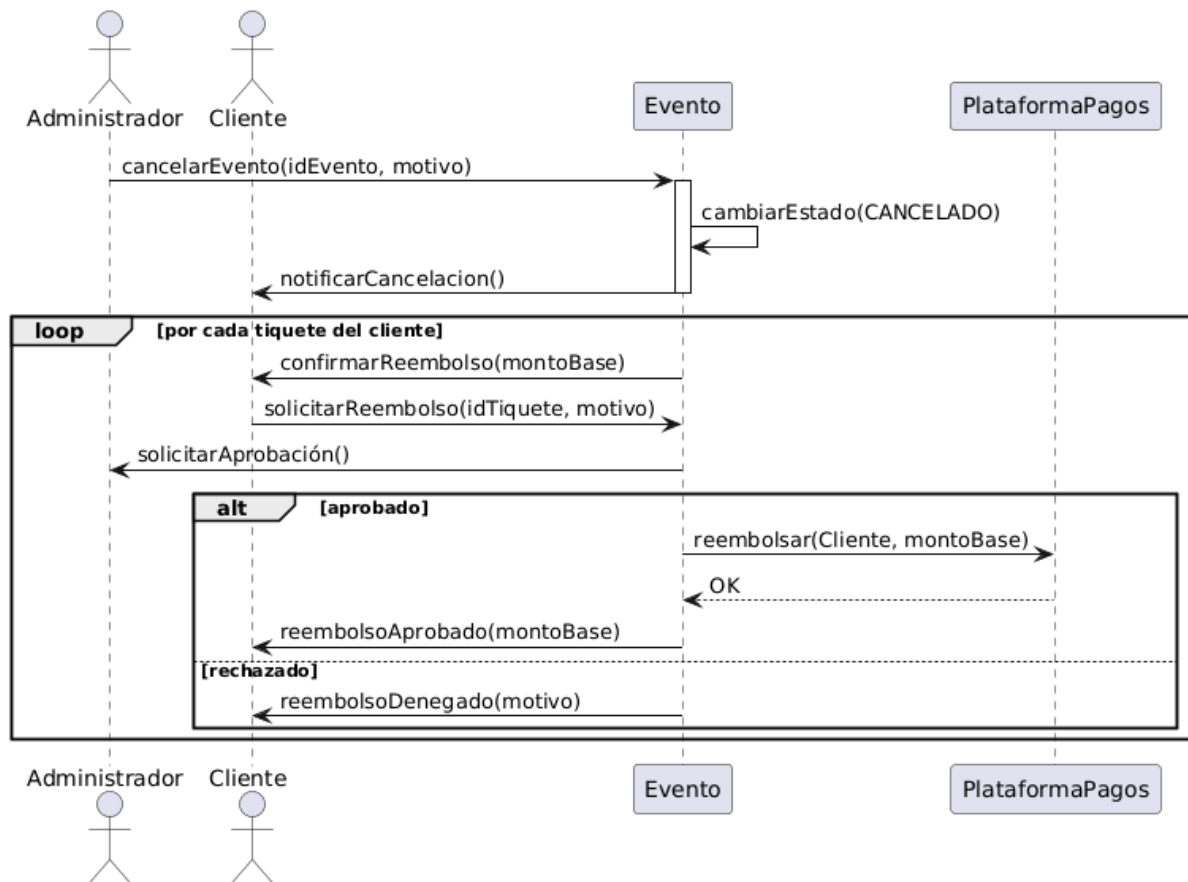


A partir de un evento ya creado (como en la secuencia anterior), la interacción es del Cliente: primero `verEvento()`, luego `seleccionarLocalidad()` y, con base en lo que necesita, `comprar(cantidad/asiento)`; dentro de un bucle `[por cada tiquete]` se evalúa **alt** según el tipo de localidad: `[localidad numerada]` → `«create» TiqueteNumerado(asiento)`, `[localidad no numerada]` → `«create» TiqueteSimple()`; finalmente el sistema `entregarTiquete(Tiquete)` al cliente y cierra con `OK (compra realizada)`.



La secuencia muestra cómo usuarios tipo Cliente se transfieren tickets: el ClienteOrigen inicia con solicitarTransferencia(idTiquete, destinoLogin), el sistema pedirCredenciales() al ClienteDestino, este enviarCredenciales(login, password) y luego se obtener(idTiquete), cargar datosTiquete y validarPropiedadYEstado(); si la validación es correcta (alt [validación OK]), el sistema transferirA(ClienteDestino), marca transferenciaRealizada y notificarRecepcion(Tiquete) al destino, retornando confirmar(OK) al origen; en caso de error de credenciales/propiedad/estado, se responde error(no se pudo transferir).

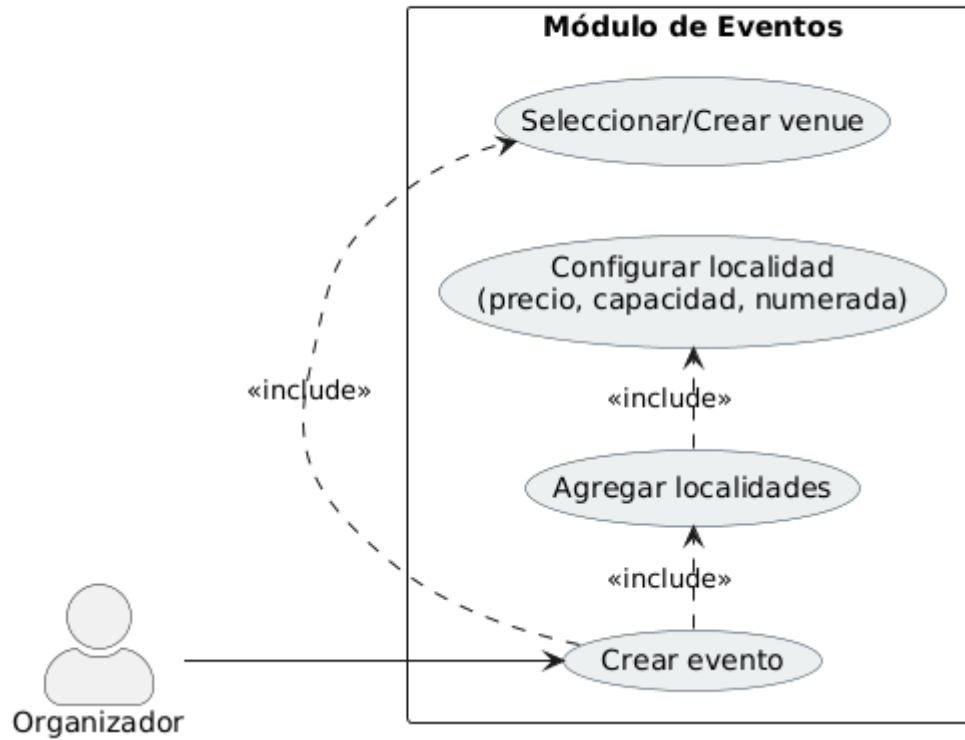
Cancelación de evento y reembolsos



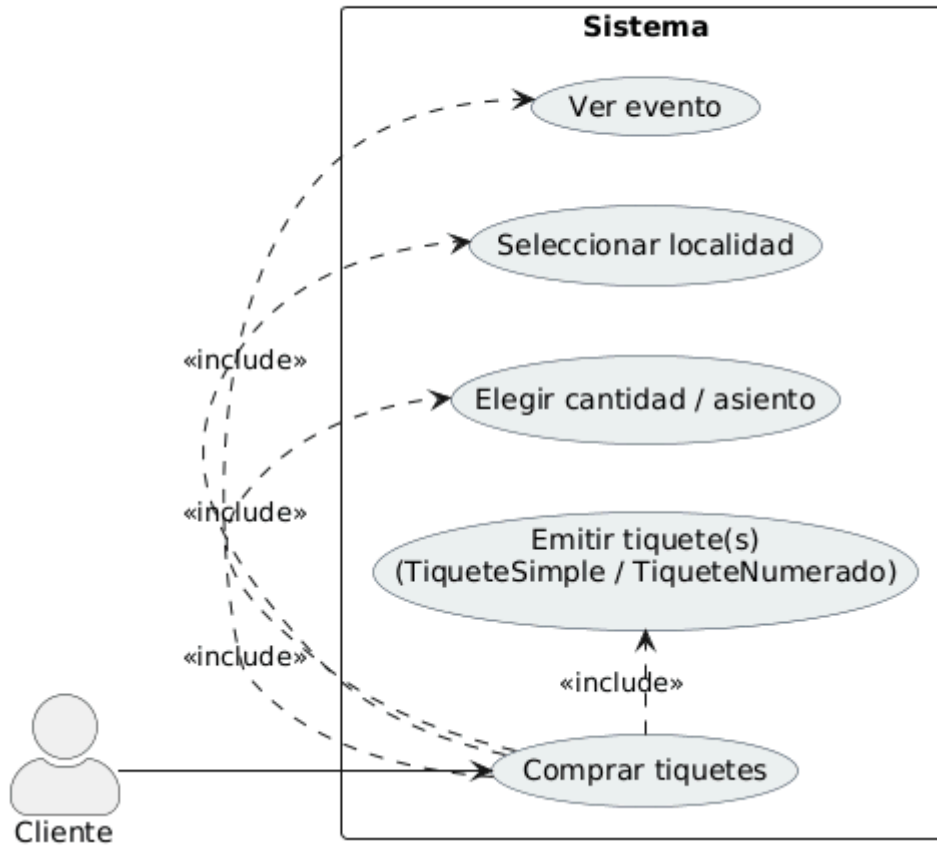
Esta secuencia corresponde a funciones del Administrador: el Administrador invoca `cancelarEvento(idEvento, motivo)`, el Evento `cambiarEstado(CANCELADO)` y `notificarCancelación()`; luego, en loop `[por cada ticket del cliente]`, se gestiona el reembolso del `montoBase` vía `PlataformaPagos` → `reembolsar(Cliente, montoBase)` (OK) y el cliente `confirmarReembolso(montoBase)`; si el cliente `solicitarReembolso(idTicket, motivo)`, el Administrador `solicitarAprobación()` y, según `alt`: `[aprobado]` la `PlataformaPagos` `reembolsar(Cliente, montoBase)` (OK) y se notifica `reembolsoAprobado(montoBase)`; `[rechazado]` se informa `reembolsoDenegado(motivo)`.

Diagramas de casos de Uso

Crear evento y asignar localidades



Compra de tiquetes



Justificación

Organización Lógica y Claridad

El sistema fue diseñado con una arquitectura modular basada en paquetes (eventos, tiquetes, usuarios), lo que promueve una separación clara de responsabilidades y facilita la mantenibilidad del código.

Cada paquete agrupa clases relacionadas con un mismo dominio funcional:

- usuarios: gestiona los distintos tipos de usuarios y sus operaciones.
- eventos: modela la estructura de los eventos, localidades y venues.
- tiquetes: maneja las reglas de negocio asociadas a las políticas de cobro y transacciones.

Esta organización lógica favorece la escalabilidad del sistema y permite integrar nuevas funcionalidades sin afectar las existentes.

Uso de Herencia y Polimorfismo

La clase abstracta Usuario agrupa atributos y comportamientos comunes.

A partir de ella se extienden tres subclases:

- Cliente: usuario que compra y transfiere tiquetes.
- Organizador: usuario que gestiona ingresos y gastos asociados a eventos.
- Administrador: usuario con permisos de gestión, configuración de políticas y control financiero.

El uso de herencia y polimorfismo facilita la extensión del sistema y evita duplicación de código, permitiendo que cada rol redefina o especialice los métodos según sus necesidades.

Encapsulamiento y Control de Errores

Se aplicó un estricto control de acceso a los atributos mediante encapsulamiento, garantizando la integridad de los datos.

Además, se incorporaron validaciones en los constructores y métodos para evitar inconsistencias, como descuentos fuera del rango permitido o montos negativos en transacciones.

Gestión de Reglas de Negocio

La clase `PoliticaCargos` centraliza las reglas de cobro por tipo de evento, permitiendo que el administrador defina cargos variables según la categoría del evento y una cuota de emisión fija.

Esto facilita la actualización de políticas sin alterar otras partes del sistema.

La clase `Transaccion` consolida las compras y reembolsos de tiquetes, manteniendo el estado y los totales calculados según las políticas vigentes.

Diseño Orientado a Casos de Uso

El diseño responde directamente a los principales casos de uso del sistema:

- Creación y cancelación de eventos.
- Generación de localidades y tiquetes.
- Compra, transferencia y reembolso de tiquetes.
- Aplicación de descuentos y ofertas.

Cada caso de uso se encuentra representado por una clase o método que refleja su comportamiento dentro del dominio del sistema.

Extensibilidad y Mantenibilidad

El diseño modular y el uso de interfaces hacen que el sistema sea fácilmente ampliable.

Por ejemplo:

- Pueden añadirse nuevos tipos de Usuario o Evento sin modificar la estructura existente.
- Nuevos servicios o políticas de precios pueden integrarse mediante la implementación de las interfaces del paquete servicios.

Esto asegura que el sistema pueda evolucionar con nuevas funcionalidades sin comprometer su estabilidad.

Diseño de la Interfaz Gráfica

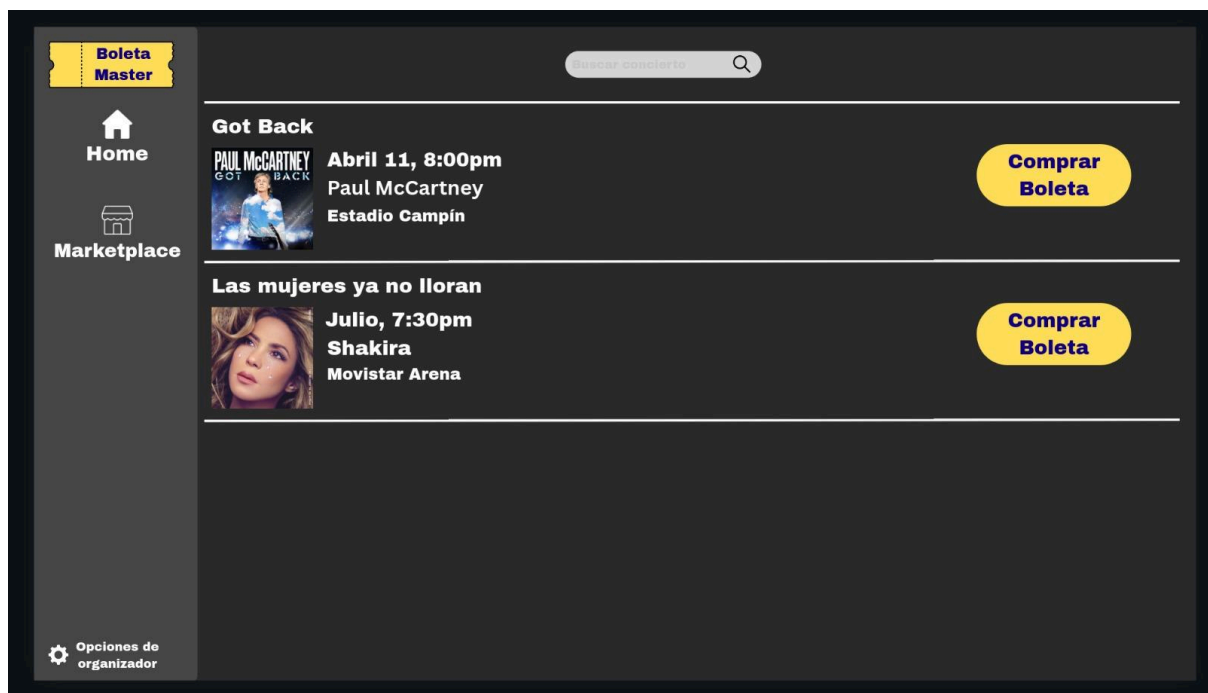
Antes de iniciar el diseño, se elaboraron los mockups. No siempre todo resulta como se espera, y a veces se presentan cambios o giros inesperados. En este caso, surgió una capa de personalización

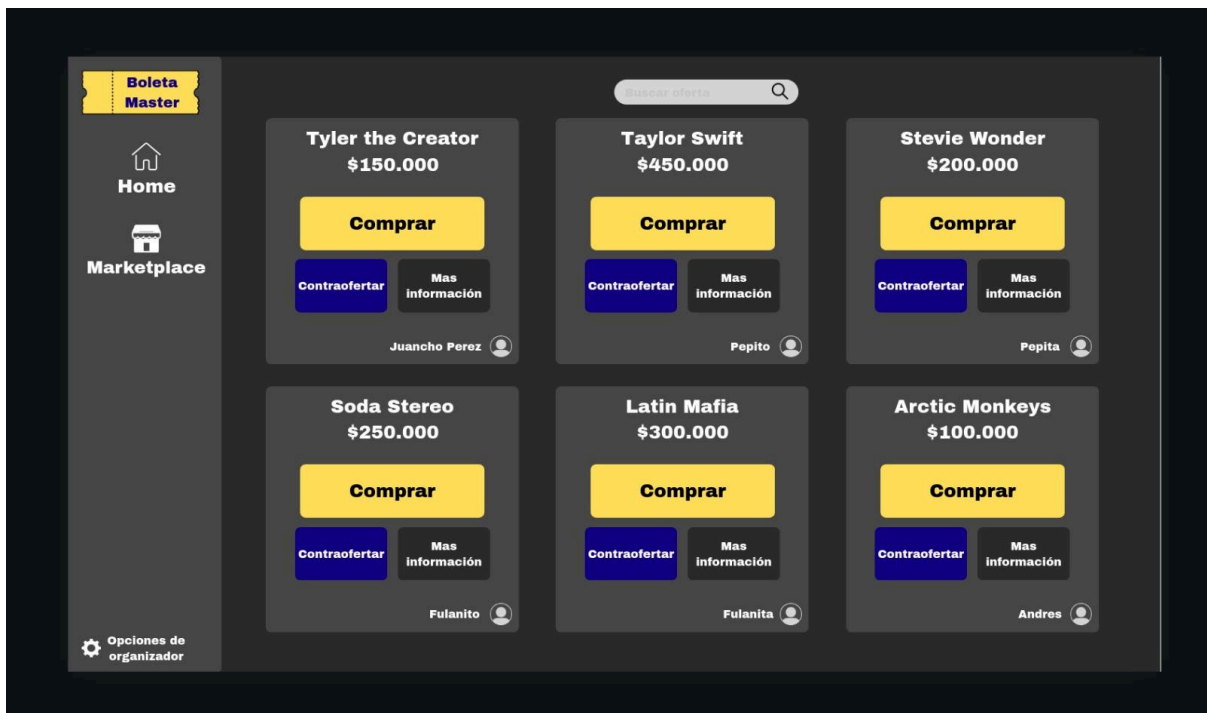
adicional: el equipo, a través de su miembro número 7, decidió crear "BoletaMaster Kakashi". Esta adición está inspirada en la icónica canción "Blue Bird" de Naruto (de Space Runner), que se mantiene sonando como un recordatorio de que tenemos la capacidad de volar hacia el cielo azul, aunque a veces no lo hacemos. Esta canción es muy significativa y se relaciona directamente con la historia del "dogordito", un ave increíble que no decidió volar. Por ello, este proyecto, "BoletaMaster Kakashi", está dedicado a él.

Por otro lado, al acceder a la interfaz, el sistema requiere un inicio de sesión (login) para autenticar el tipo de usuario (cliente, administrador u organizador), ya que cada uno tiene roles y permisos diferentes. Una vez dentro, la interfaz es clara y limpia, destacando las pestañas laterales: Home, Mis Boletas, Marketplace, Organizador y Admin. Las dos últimas pestañas solo son accesibles para usuarios con roles de administrador u organizador, no para clientes.

Para la generación del código QR, se integró una librería externa de alto soporte y muy consolidada: ZXing. Esta herramienta es concisa, tiene releases desde 2013 y permite la creación de códigos QR en imágenes PNG en tiempo real, basándose en el ID único de cada tiquete.

Mockups





Implementación

